

AI Native Agent Team Curriculum

AI Native Agent Team Curriculum

■ 1인 창업자를 위한 Agent Team 운영 과정

- 컨텍스트 자산을 만들고
- 반복 업무를 Skill로 고정하고
- Agent Team과 승인 큐로 운영한다
- 마지막에는 마케팅 루프까지 달는다

8주 후 목표: 외주 결과물이 아니라 직접 운영 가능한 자동화 체계를 가진다.



이 과정의 목표는 자동화 외주가 아니라 운영 역량입니다

수강자는 8주 후 다음 상태에 도달해야 합니다.

- **컨텍스트 자산화**: 사업 지식, 문체, 의사결정을 Markdown으로 관리
- **코드 자산화**: Claude Code 산출물을 Git으로 되돌릴 수 있게 관리
- **Agent Team 운영**: 8개 이하 Agent와 승인 큐를 직접 점검
- **매출 업무 자동화**: 타로 상담, 콘텐츠, 마케팅 루프를 단계적으로 연결

완료 기준: 본인이 Agent 1개 또는 Skill 1개를 문서만 보고 추가할 수 있다.

목차: 2트랙 8주 과정으로 Agent Team 운영 역량을 만든다

이 과정의 목표는 자동화 외주가 아니라 운영 역량입니다 > 목차: 2트랙 8주 과정으로 Agent Team 운영 역량을 만든다

■ Track A — 기초 지식

1~2주차 · EDU

- 모듈 1~7
- 프레임워크를 이해할 눈 만들기
- 터미널, Markdown, Git, API, 보안
- 라이선스와 라이브러리 검증

비율: 이론 40 / 실습 40 / 과제 20

■ Track B — 실전 프레임워크

3~8주차 · BUILD

- 모듈 8~16
- 본인 Agent Team을 직접 만들고 고치기
- 컨텍스트, Skill, Agent, 승인 큐
- Conductor와 마케팅 루프

종착점: 8개 Agent Team을 스스로 운영·확장한다.

핵심 원칙은 파인튜닝이 아니라 컨텍스트 자산화입니다

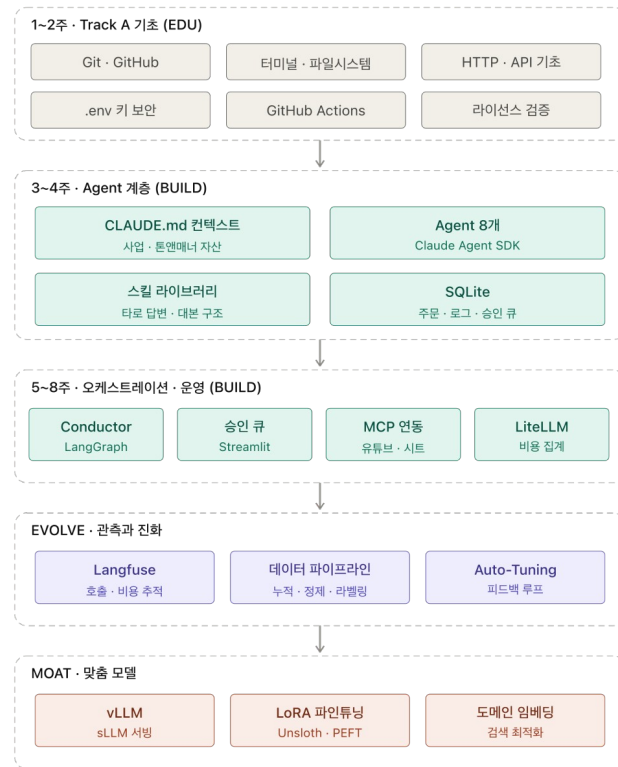
목차: 2트랙 8주 과정으로 Agent Team 운영 역량을 만든다 > Track B — 실전 프레임워크 > 핵심 원칙은 파인튜닝이 아니라 컨텍스트 자산화입니다

■ 학습시킨다는 말의 실무 해석

고객이 원하는 "AI 학습"은 모델 재훈련이 아닙니다. 반복 가능한 업무 지식을 파일과 규칙으로 쌓아 Agent가 같은 기준으로 판단하게 만드는 일입니다.

기본 저장소 구조

- `CLAUDE.md`: 사업 정체성과 원칙
- `context/`: 고객, 상품, 가격, 채널
- `skills/`: 반복 업무 절차
- `logs/`: 실행 결과와 성과 지표



자산화 단계는 프롬프트에서 모델 생성까지 깊어집니다

자산화는 프롬프트에서 모델 생성까지 깊어진다

앞 단계가 축적될수록 뒤 단계의 자동화 투자 효율이 높아진다



운영 기준 프롬프트와 컨텍스트가 흔들리면 에이전트, 백엔드, 모델 생성 투자는 모두 재작업이 된다.

계층도는 2개월 과정과 미래 확장 단계를 함께 보여줍니다

계층도는 2개월 과정과 미래 확장 단계를 함께 보여준다

점선 구역 하나가 학습 단계 하나이며, 색은 패키지 범위를 encoding한다

EDU

회색 기초 구역

Git, Node.js, Python, Docker, API, 라이선스 검증을 배워 Track B를 이해할 눈을 만든다.

BUILD

Agent 계층

Claude Agent SDK, LangGraph, MCP, SQLite, Streamlit으로 8개 이하 Agent Team을 만든다.

세 번째 구역까지 내려오면 BUILD KPI: 8개 Agent 정상 작동 + 승인 큐 운영.

EVOLVE

관측과 확장 구역

Langfuse, PostgreSQL, pgvector, Chroma, Temporal은 BUILD에서 쌓인 로그와 데이터를 아래로 흘려보낸다.

MOAT

모델 생성 구역

vLLM, Unsloth, PEFT, sentence-transformers는 지금 만드는 것이 아니라, 운영 데이터가 준비시켜주는 미래 단계다.

고객 설명 문장

“이 그림에서 점선 세 칸을 내려오는 것이 2개월 과정입니다.”

아래 두 구역은 지금 만드는 것이 아니라, 위 시스템의 로그와 데이터가 열어주는 다음 단계입니다.

전체 아키텍처는 Conductor와 8개 이하 Agent로 제한합니다

계층도는 2개월 과정과 미래 확장 단계를 함께 보여줍니다 > 전체 아키텍처는 Conductor와 8개 이하 Agent로 제한합니다

■ 운영 중심 Agent

Conductor

- 업무 라우팅
- 일일 브리핑
- 승인 큐 관리
- Agent 간 상태 전달

분석 Agent

- 사업기획 참모
- 광고 성과분석
- 유튜브 기획총괄

■ 외부 발신 Agent

제작 Agent

- 광고 소재
- 유튜브 작가
- 댓글 운영
- 스톱 콘텐츠
- 타로 상담

공통 원칙

- 외부 발신은 초안까지만 자동화
- 승인 후 게시 또는 발송
- 자동 좋아요, 자동 리포스트는 제외

Track A

전체 아키텍처는 Conductor와 8개 이하 Agent로 제한합니다 > 외부 발신 Agent > Track A

기초 지식 2주

1주차는 파일, 문서, Git으로 안전한 작업 기반을 만듭니다

Track A > 1주차는 파일, 문서, Git으로 안전한 작업 기반을 만듭니다

■ Module 1

터미널과 파일 시스템

- `cd`, `ls`, `mkdir`, `cat`, `mv`
- 절대경로와 상대경로
- 사업 폴더 구조 생성

과제: `business/tarot`,
`business/youtube`,
`business/context` 만들기

■ Module 2

Markdown과 문서 자산화

- 사업 개요 작성
- 톤앤매너 예시 축적
- 컨텍스트 유무 A/B 비교

과제: 잘된 콘텐츠 10개를 정리

■ Module 3

Git과 GitHub

- 커밋은 세이프포인트
- 브랜치는 실험 공간
- GitHub는 백업과 이력

과제: 작업마다 커밋 10개 이상 남기기

2주차는 외부 연동과 운영 안전성을 배웁니다

1주차는 파일, 문서, Git으로 안전한 작업 기반을 만듭니다 > Module 3 > 2주차는 외부 연동과 운영 안전성을 배웁니다

■ Module 4

HTTP와 API

- 요청과 응답
- GET, POST
- JSON, API Key, Bearer Token

과제: 유튜브 댓글 10개를 API로 저장

■ Module 5

환경변수와 키 보안

- `.env`
- `.gitignore`
- 키 유출 방지

과제: 하드코딩된 API 키 0개 만들기

■ Module 6-7

실행 환경과 오픈소스 검증

- 로컬과 서버의 차이
- GitHub Actions
- MIT, Apache, GPL 구분

과제: 매일 오전 자동 실행 스크립트 만들기

Track A의 검수 기준은 실제로 돌아가는 작은 자동화입니다

2주차는 외부 연동과 운영 안전성을 배웁니다 > Module 6-7 > Track A의 검수 기준은 실제로 돌아가는 작은 자동화입니다

- 타로사이트 또는 업무 자동화 코드가 Git으로 관리된다.
- 사업 개요와 문체 예시가 Markdown 컨텍스트로 정리된다.
- API Key는 환경변수 파일로 분리되고 GitHub에 올라가지 않는다.
- 매일 1회 실행되는 자동화가 실제로 작동한다.

이 단계의 목적: AI에게 과감하게 일을 시켜도 되돌릴 수 있는 작업 안전망을 만든다.

Track B

Track A의 검수 기준은 실제로 돌아가는 작은 자동화입니다 > Track B

실전 프레임워크 6주

실습은 빈 폴더에서 Agent Team까지 하나씩 조립합니다

실습은 빈 폴더에서 Agent Team까지 하나씩 조립한다

각 랩은 다음 랩의 입력물이 되므로, 수강자는 완성물이 쌓이는 경험을 한다



검수 기준

각 랩의 산출물이 다음 랩의 입력물로 연결되어야 한다.

최종 검수는 "8개 이하 Agent 정상 작동 + 승인 큐 1주일 운영 + 새 Skill 단독 추가"다.

실습 repo는 프롬프트와 커밋 로그가 진행표입니다

실습은 빈 폴더에서 Agent Team까지 하나씩 조립합니다 > 실습 repo는 프롬프트와 커밋 로그가 진행표입니다

■ GitHub에서 내려받고 시작

- 공개 repo: github.com/baryonlabs/edu-agent-team-lab-repo-practice
- 시작 명령: [git clone https://github.com/baryonlabs/edu-agent-team-lab-repo-practice.git](https://github.com/baryonlabs/edu-agent-team-lab-repo-practice.git)
- 교재 PDF와 단계 가이드는 repo 안의 [docs/](#)에서 확인한다.

■ 수강자 작업 방식

- [prompts/01-repo-skeleton.md](#)부터 순서대로 실행한다.
- 각 프롬프트가 요구한 산출물을 만든다.
- 실행 확인 후 `git commit`으로 단계를 닫는다.
- [COMMIT_LOG.md](#)와 `git log --oneline --reverse`로 진행 상태를 확인한다.

■ 확인해야 할 문서

- [README.md](#): 시작 방법과 실행 명령
- [docs/STAGE_GUIDE.md](#): 단계별 코드 설명
- [COMMIT_LOG.md](#): 1~6단계 커밋 메시지와 해시

시작 프롬프트는 repo 다운로드부터 커밋까지 이어지게 씁니다

실습 repo는 프롬프트와 커밋 로그가 진행표입니다 > 확인해야 할 문서 > 시작 프롬프트는 repo 다운로드부터 커밋까지 이어지게 씁니다

■ Claude Code에 넣는 첫 프롬프트

```
GitHub에서 실습 repo를 내려받아 시작한다.  
repo 주소는 https://github.com/baryonlabs/edu-agent-team-lab-repo-practice 이다.  
README.md와 docs/STAGE_GUIDE.md를 읽고,  
prompts/01-repo-skeleton.md부터 prompts/06-conductor-logging.md까지 순서대로 진행한다.  
각 단계가 끝나면 확인 명령을 실행하고, 지정된 커밋 메시지로 commit한다.
```

■ 수강자가 직접 치는 시작 명령

- `git clone https://github.com/baryonlabs/edu-agent-team-lab-repo-practice.git`
- `cd edu-agent-team-lab-repo-practice`
- `git log --oneline --reverse`

▶ 핵심: 프롬프트가 다음 행동을 만들고, Git history가 학습 과정을 증명한다.

실습 코드는 입력, 생성, 검수, 브리핑 흐름으로 읽습니다

시작 프롬프트는 repo 다운로드부터 커밋까지 이어지게 씁니다 > 수강자가 직접 치는 시작 명령 > 실습 코드는 입력, 생성, 검수, 브리핑 흐름으로 읽습니다

■ 코드 실행 흐름

1. 입력과 저장

- `backend/seed_order.py`: 샘플 주문 생성
- `backend/schema.sql`: `orders`, `drafts`, `approvals`, `run_logs`
- `backend/db.py`: SQLite 연결과 로그 기록

2. 생성과 검수

- `agents/tarot_agent.py`: 주문을 답변 초안으로 변환
- `approval_queue/app.py`: 승인, 반려 상태 변경
- `conductor/daily_briefing.py`: 운영 상태 요약

■ 수업에서 읽는 순서

먼저 보는 것

- `prompts/*.md`: 무엇을 만들라고 지시하는가
- `CLAUDE.md`: Agent가 지켜야 할 기준
- `skills/tarot-response.md`: 반복 답변 절차

나중에 보는 것

- DB 테이블 간 관계
- 상태값 `new`, `drafted`, `pending`, `approved`, `rejected`
- `run_logs`가 관측과 개선 데이터가 되는 방식

Lab 1: 저장소 골격은 코드가 쌓일 위치를 먼저 고정합니다

실습 코드는 입력, 생성, 검수, 브리핑 흐름으로 읽습니다 > 수업에서 읽는 순서 > Lab 1: 저장소 골격은 코드가 쌓일 위치를 먼저 고정합니다

■ 실습 목표

빈 폴더를 **Agent Team 실습 repo**로 바꾸고, 이후 단계가 들어갈 위치를 확정한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/01-repo-skeleton.md`
- 핵심 파일: `.gitignore`, `README.md`
- 진행표: `COMMIT_LOG.md`
- 코드 폴더: `agents/`, `backend/`, `approval_queue/`
- 지식 폴더: `context/`, `skills/`, `conductor/`

■ 알아야 할 정보

- 폴더는 장식이 아니라 **역할 경계**다.
- DB와 로그는 `.gitignore`로 제외한다.
- 단계가 끝나면 `git commit -m "step 1: create repo skeleton"`으로 닫는다.

확인 명령: `find . -maxdepth 2 -type d | sort`

Lab 2: 컨텍스트 팩은 Agent가 따라야 할 사업 기준입니다

Lab 1: 저장소 골격은 코드가 쌓일 위치를 먼저 고정합니다 > Lab 2: 컨텍스트 팩은 Agent가 따라야 할 사업 기준입니다

■ 실습 목표

사업 개요, 문체, 금지사항을 Markdown으로 고정해 Agent가 같은 기준으로 답하게 한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/02-context-pack.md`
- 핵심 파일: `CLAUDE.md`
- 참조 파일: `context/business-overview.md`,
`context/tone-examples.md`

■ 알아야 할 정보

- 파인튜닝 전에 먼저 **컨텍스트 자산화**를 한다.
- 좋은 예시와 금지 예시는 답변 품질을 직접 바꾼다.
- 실제 고객 실명, API Key, 민감정보는 넣지 않는다.

확인 명령: `sed -n '1,120p' CLAUDE.md`

Lab 3: Skill은 반복 프롬프트를 업무 절차로 바꿉니다

Lab 2: 컨텍스트 팩은 Agent가 따라야 할 사업 기준입니다 > Lab 3: Skill은 반복 프롬프트를 업무 절차로 바꿉니다

■ 실습 목표

타로 답변 구조를 문서화해 Agent가 매번 같은 순서로 초안을 만들게 한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/03-tarot-skill.md`
- 핵심 파일: `skills/tarot-response.md`
- 출력 순서: 카드 해석
 - 종합 흐름 → 현실 조언 → 확인 질문

■ 알아야 할 정보

- Skill은 "좋은 답변을 한 번 쓰기"가 아니라 **반복 기준을 남기는 일**이다.
- 좋은 예시 2개와 피해야 할 예시 1개를 넣는다.
- 다음 단계의 `tarot_agent.py`가 이 구조를 코드로 흉내 낸다.

확인 명령: `sed -n '1,160p' skills/tarot-response.md`

Lab 4: 단일 Agent는 주문을 초안과 승인 대기로 바꿉니다

Lab 3: Skill은 반복 프롬프트를 업무 절차로 바꿉니다 > Lab 4: 단일 Agent는 주문을 초안과 승인 대기로 바꿉니다

■ 실습 목표

샘플 주문을 SQLite에 넣고, `tarot_agent.py`가 답변 초안을 생성하게 한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/04-single-agent.md`
- DB 파일: `backend/schema.sql`, `backend/db.py`
- 실행 파일: `backend/seed_order.py`,
`agents/tarot_agent.py`

■ 알아야 할 정보

- `orders`는 입력, `drafts`는 초안, `approvals`는 검수 상태다.
- `run_logs`는 나중에 Langfuse로 확장될 관측 지점이다.
- 현재 예제는 LLM 호출 대신 deterministic draft로 구조를 먼저 검증한다.

확인 명령: `python backend/seed_order.py && python agents/tarot_agent.py`

Lab 5: 승인 큐는 외부 발신 전에 사람이 보는 안전장치입니다

Lab 4: 단일 Agent는 주문을 초안과 승인 대기로 바꿉니다 > Lab 5: 승인 큐는 외부 발신 전에 사람이 보는 안전장치입니다

■ 실습 목표

생성된 초안을 바로 보내지 않고 Streamlit 화면에서 승인 또는 반려하게 한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/05-approval-queue.md`
- 핵심 파일: `approval_queue/app.py`
- 상태 변경: `pending` → `approved` 또는 `rejected`

■ 알아야 할 정보

- "내 이름으로 나가는 것은 내가 본다"가 운영 원칙이다.
- `set_status()`는 `drafts`와 `approvals`를 함께 업데이트한다.
- 버튼 클릭은 `run_logs`에 남아 다음 브리핑의 근거가 된다.

확인 명령: `streamlit run approval_queue/app.py`

Lab 6: Conductor는 Agent Team의 운영 상태를 한 장으로 요약합니다

Lab 5: 승인 큐는 외부 발신 전에 사람이 보는 안전장치입니다 > Lab 6: Conductor는 Agent Team의 운영 상태를 한 장으로 요약합니다

■ 실습 목표

개별 Agent 실행 결과를 모아 승인 대기, 승인, 반려 건수와 최근 로그를 브리핑한다.

■ 프롬프트와 코드

- 프롬프트: `prompts/06-conductor-logging.md`
- 핵심 파일: `conductor/daily_briefing.py`
- 참조 데이터: `drafts`, `approvals`, `run_logs`

■ 알아야 할 정보

- Conductor는 개별 Agent를 대체하지 않고 **상태를 수집하고 판단을 돕는다**.
- 일일 브리핑은 GitHub Actions나 cron으로 자동화할 수 있다.
- 로그가 쌓이면 EVOLVE와 MOAT 단계의 학습 데이터가 된다.

확인 명령: `python conductor/daily_briefing.py`

실습 정리: 프레임워크는 산출물이 이어질 때 자산이 됩니다

Lab 6: Conductor는 Agent Team의 운영 상태를 한 장으로 요약합니다 > 실습 정리: 프레임워크는 산출물이 이어질 때 자산이 됩니다

- Lab 1의 저장소 골격은 이후 코드가 들어갈 위치를 정한다.
 - Lab 2의 컨텍스트 팩은 Lab 3의 Skill 품질을 결정한다.
 - Lab 3의 Skill은 Lab 4의 단일 Agent가 따르는 답변 구조가 된다.
 - Lab 4의 초안은 Lab 5의 승인 큐로 들어간다.
 - Lab 6의 브리핑은 실행 로그를 운영 판단으로 바꾼다.
- ▶ 핵심 문장: **프롬프트를 잘 쓰는 교육이 아니라, 돌아가는 업무 시스템을 하나씩 쌓는 교육입니다.**

3-4주차는 컨텍스트와 첫 매출 Agent를 완성합니다

실습 정리: 프레임워크는 산출물이 이어질 때 자산이 됩니다 > 3-4주차는 컨텍스트와 첫 매출 Agent를 완성합니다

■ Module 8-10

프로젝트 구조와 Skill

- `agents/`, `context/`, `skills/`, `logs/` 4분할
- `CLAUDE.md`로 사업 기준 정리
- 타로 답변 Skill 제작
- 유튜브 대본 구조 Skill 제작

성과물: 반복 업무가 프롬프트가 아니라 재사용 가능한 Skill로 남는다.

■ Module 11

타로 상담 Agent

- 주문 접수
- 카드 해석 초안
- 종합 조언
- 승인 대기
- 발송

성과물: 실제 주문 3건을 Agent로 처리하고 수동 대비 시간을 측정한다.

5-6주차는 연동, 콘텐츠, 승인 큐로 사고를 줄입니다

3-4주차는 컨텍스트와 첫 매출 Agent를 완성합니다 > Module 11 > 5-6주차는 연동, 콘텐츠, 승인 큐로 사고를 줄입니다

■ Module 12-13

외부 시스템과 콘텐츠 Agent

- YouTube API
- Google Sheets
- 유튜브 작가 Agent
- 댓글 초안 Agent
- 팩트체크 Skill

성과물: 대본 1편과 댓글 초안을 Agent로 생산한다.

■ Module 14

Human-in-the-loop 승인 큐

- 타로 답변 승인
- 댓글 승인
- 스레드 게시 승인
- 수정과 반려 기록

운영 원칙: 내 이름으로 나가는 것은 내가 본다.

7-8주차는 Conductor와 마케팅 루프로 운영 체계를 닫습니다

5-6주차는 연동, 콘텐츠, 승인 큐로 사고를 줄입니다 > Module 14 > 7-8주차는 Conductor와 마케팅 루프로 운영 체계를 닫습니다

■ Module 15

Conductor

- Agent 상태 수집
- 승인 대기 건수 집계
- 오늘의 우선순위 정리
- 오전 8시 일일 브리핑

성과물: 하루 운영을 한 장의 브리핑으로 시작한다.

■ Module 16

Marketing Loop

- 광고 카피와 소재 프롬프트 생성
- 성과 수집과 개선안 도출
- 다음 소재에 반영

성과물: 생성, 집행, 분석, 개선이 한 번 이상 회전한다.



주차 설계는 매출 직결 업무를 먼저 완성하도록 배치합니다

7-8주차는 Conductor와 마케팅 루프로 운영 체계를 담습니다 > Module 16 > 주차 설계는 매출 직결 업무를 먼저 완성하도록 배치합니다

■ 8주 운영 흐름

1. 1-2주차: 터미널, Markdown, Git, API, 보안, 실행 환경
2. 3-4주차: 저장소 구조, [CLAUDE.md](#), Skill, 타로 Agent
3. 5-6주차: 외부 연동, 콘텐츠 Agent, 승인 큐
4. 7-8주차: Conductor, 일일 브리핑, 마케팅 루프

■ 배치 이유

먼저 만드는 것

- 되돌릴 수 있는 작업 환경
- 컨텍스트 저장소
- 타로 상담 Agent

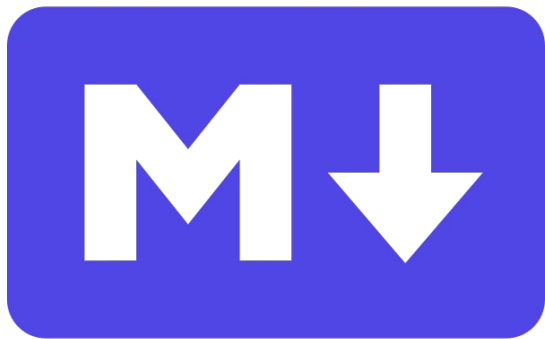
나중에 붙이는 것

- 콘텐츠 팀
- 광고 루프

오픈소스는 계층별로 검증하고 채택합니다

주차 설계는 매출 직결 업무를 먼저 완성하도록 배치합니다 > 배치 이유 > 오픈소스는 계층별로 검증하고 채택합니다

■ 문서 자산화



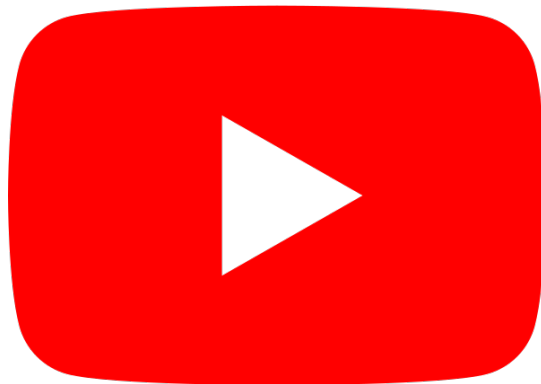
- Markdown: 컨텍스트 표준 포맷
- [CLAUDE.md](#): 사업 기준과 금지사항

■ 코드 자산화



- Git: 되돌릴 수 있는 세이프포인트
- GitHub: 백업과 이력 관리

■ 외부 연동



- YouTube API: 댓글과 채널 데이터
- Google Sheets: 주문과 성과 대장

오픈소스 통합표: 고객이 실제로 손으로 만지는 핵심 도구

라이선스와 학습 모듈을 함께 보며, 모듈 7의 라이선스 검증 교보재로 사용한다

계층	도구	용도	라이선스	모듈	패키지
L0 개발 기반	Git, Node.js, Python	코드 자산화, 런타임, 스크립트	GPL-2.0, MIT, PSF	1, 3	EDU
L0 개발 기반	Docker Engine, uv, pnpm	환경 패키징과 패키지 관리	Apache-2.0, MIT	6	EDU
L1 LLM 게이트웨이	LiteLLM, Ollama	모델 호출 단일화, 비용 로깅, 로컬 추론	MIT	15, 소개	BUILD
L2 Agent	Claude Agent SDK	CLAUDE.md, Skill, Subagent 기반 개별 Agent	MIT	9~11	BUILD
L2 Agent	LangGraph, Pydantic AI	Conductor, 승인 큐, 구조화 출력	MIT	11, 14~15	BUILD
L3 도구 연결	MCP, fetch, playwright-mcp	외부 도구 표준 연결, 웹 조회, 브라우저 자동화	MIT	12~13	BUILD
L3 도구 연결	google-api-python-client	YouTube Data API, Google Sheets 연동	Apache-2.0	4, 12	EDU/BUILD
L4 데이터	SQLite	주문, 로그, 승인 큐 저장	Public Domain	8, 11	BUILD
L5 스케줄링	GitHub Actions	cron 자동 실행, 일일 브리핑	워크플로우 자유	6, 15	EDU/BUILD
L6 UI	Streamlit, Gradio, Next.js	승인 큐, 검수 UI, 고객용 사이트	Apache-2.0, MIT	14, 보류	BUILD
L7 관측	Langfuse	LLM 호출 추적, 비용, 품질 평가	MIT core	16	BUILD→EVOLVE

표준 스택: Claude Agent SDK + LangGraph + MCP + SQLite + Streamlit + GitHub Actions + Langfuse + Docker Compose

확장 도구는 지금 구축하지 않고 필요 시점을 가르칩니다

오픈소스 통합표: 소개만 하는 확장 도구와 라이선스 주의점

고객이 당장 구축하지 않는 도구는 “언제 필요해지는지”만 설명한다

계층	도구	쓰는 시점	라이선스	패키지	주의
L2 Agent 참고	CrewAI, AutoGen(AG2)	비교 소개. 이 케이스 표준은 아님	MIT / Apache-2.0	—	도구 과잉 방지
L4 벡터 검색	PostgreSQL, pgvector, Chroma	콘텐츠 수백 건 이후 검색 품질이 병목일 때	PostgreSQL / Apache	EVOLVE	초기 도입 금지
L5 워크플로우	n8n	라이선스 판별 실습 사례	Sustainable Use	—	오픈소스 아님
L5 워크플로우	Temporal	실패 복구와 장기 실행이 중요해질 때	MIT	EVOLVE 이후	지금은 과함
L7 관측	OpenTelemetry	표준 계측 개념 소개	Apache-2.0	EVOLVE	개념만
L8 배포	Docker Compose, Coolify, Caddy	인수인계, 셀프호스팅, HTTPS 운영	Apache-2.0	BUILD/EVOLVE	운영 단계
L9 MOAT	vLLM, Unsloth, PEFT, 임베딩	도메인 데이터가 충분히 쌓인 뒤	Apache-2.0	MOAT	소개만

읽는 법

총 34개 도구 중 고객이 실제로 손으로 만지는 것은 15개 내외다.

나머지는 “언제 필요해지는지”를 아는 수준으로만 다룬다.

n8n은 오픈소스처럼 보이지만 아닌 도구를 판별하는 모듈 7 교보재로 쓴다.

운영 원칙은 자동화 범위를 명확히 제한합니다

확장 도구는 지금 구축하지 않고 필요 시점을 가르칩니다 > 운영 원칙은 자동화 범위를 명확히 제한합니다

- 외부 발신물은 초안 생성 후 승인 큐를 통과한다.
- Meta, YouTube 계정 제재 위험이 큰 자동 활동은 범위에서 제외한다.
- 로그는 비용, 품질, 반려 사유, 성과 지표를 함께 남긴다.
- Agent 수는 8개 이하로 유지하고, 역할 분리는 먼저 Skill과 프롬프트로 해결한다.
- 새 자동화는 매출, 시간 절감, 리스크 감소 중 하나를 명확히 증명해야 한다.

최종 산출물

8주 후 수강자는 외주 결과물을 받는 것이 아니라 직접 운영 가능한 Agent Team 을 가진다.

Context Asset → Skill → Agent → Approval Queue → Conductor → Marketing Loop
완료 기준은 단순합니다.
문서를 보고 Agent 또는 Skill 하나를 스스로 추가할 수 있어야 합니다.